Autonomous Scientific Discovery System

Capstone Project Blueprint: LLM Agents for Hardware-Grounded Experimental Design

1. Project Overview: The Closed-Loop AI Scientist

This project proposes a benchtop-scale "Self-Driving Laboratory" (SDL). Traditional lab automation executes pre-programmed instructions; this system pairs physical hardware with LLM agents capable of dynamically reasoning, formulating hypotheses, triggering physical experiments, analyzing resulting data, and generating a written scientific report autonomously.

> ***The Core Loop:*** *Give the agent a high-level prompt (e.g., "Investigate the relationship between the angle of a polarized filter and transmitted light intensity"). The agent hypothesizes, actuates hardware, analyzes data via active learning, and iterates until the physical phenomenon is mapped and documented.*

2. Technology & Hardware Stack

2.1 Software & AI Architecture

- **LLM Engine:** GPT-4o or Claude 3.5 Sonnet (required for advanced tool-calling and code generation capabilities).
- **Agent Framework:** Microsoft AutoGen or LangChain. AutoGen is recommended for creating distinct agent personas (Planner, Hardware Executor, Analyst, Writer).
- **Active Learning / Math:** Scikit-Optimize (`skopt`) or BoTorch for Bayesian Optimization, acting as a tool for the LLM to determine the most statistically valuable next physical measurement.
- **Hardware Interface:** Python `pyserial` to bridge high-level LLM tool calls with low-level microcontroller execution.

- **Reporting:** `matplotlib` for data visualization and `pylatex` for autonomous generation of formatted PDF research papers.

2.2 Hardware Architecture

- **Controller:** Arduino Uno or ESP32 running C++ firmware to execute serial commands (e.g., `MOVE 45`) and return sensor state.
- **Actuation:** NEMA 17 Stepper Motors (with A4988 drivers) for precision, or SG90 servos for a simpler, lower-cost alternative.
- **Sensors:** I2C-based digital sensors, such as the Adafruit TSL2591 (high-precision light sensor) or MLX90614 (IR temperature sensor).

3. Suggested Physical Experiments

The chosen experiment must present a clear mathematical relationship for the agent to "discover" via active learning:

- **Malus's Law (Light & Polarization):** A servo rotates a polarized film in front of a light source, while a stationary sensor measures intensity. The agent discovers the cosine-squared relationship.
- **The Inverse Square Law (Radiation/Light):** A stepper motor drives a sensor along a linear rail away from a stationary source. The agent maps intensity decay over distance.

4. Academic & Research Context

This project grounds its methodology in recent breakthroughs in autonomous scientific discovery. Key reference points include:

- **"The AI Scientist" (Sakana AI, 2024):** Demonstrated open-ended scientific discovery, brainstorming, and automated paper generation in software. This capstone adapts this concept to physical hardware.
- **"Coscientist" (Boiko et al., Nature, 2023):** Showcased LLMs orchestrating robotic chemistry labs, proving that foundation models can plan and execute physical iterative experiments.
- **Self-Driving Laboratories (SDLs):** The broader paradigm of combining high-throughput automated hardware with Bayesian optimization in a closed loop.

5. Development Pipeline

A strict bottom-up approach is required to prevent hardware damage from AI hallucinations. The "Body" must be robust before the "Brain" is attached.

Phase 1: The Body (Hardware Foundation & Serial Control)

Goal: Build the physical rig and ensure reliable PC command execution without AI.

1.1 Physical Assembly: Mount actuators and sensors to ensure precise manipulation of the physical relationship.

1.2 Microcontroller Firmware: Write C++ code to listen for serial strings and return parseable data (e.g., `STATUS: OK, DATA: 450.2`).

1.3 Python Serial Wrapper: Write a Python script using `pyserial` to establish two-way communication with the microcontroller.

Phase 2: The Nervous System (Tool Abstraction & Safety Boundaries)

Goal: Convert the Python serial script into a safe API for the LLM to call.

2.1 Python API Development: Create discrete functions like `move_motor(angle)` and `get_reading()`.

2.2 Hardcoding Safety Limits: Implement strict bounds checking inside the Python functions to prevent the LLM from executing physically damaging commands (e.g., out-of-bounds angles).

2.3 Human-in-the-Loop Mock Test: Manually test the API through a Python loop to verify actuation, data logging, and safety constraints.

Phase 3: The Brain (Agentic Intelligence & Active Learning)

Goal: Construct AI agents and provide access to the hardware API.

3.1 Framework Setup: Initialize AutoGen or LangChain. Bind the Python API functions as executable tools for the LLM.

3.2 Bayesian Optimization Integration: Integrate Scikit-Optimize or BoTorch as an LLM tool to handle complex numerical optimization and dictate the next most valuable experimental parameter.

3.3 Multi-Agent Orchestration: Define specialized agents: a Planner Agent, an Executor Agent, and an Analyst Agent.

Phase 4: The Final Run (Autonomous Loop & Report Generation)

Goal: Execute a full, unassisted experimental loop.

4.1 The Closed-Loop Test: Provide the Planner Agent with a high-level prompt and allow the system to plan, execute, optimize, and iterate autonomously.

4.2 Writer Agent Setup: Pass the final dataset to a Writer Agent responsible for generating data visualizations (matplotlib).

4.3 Document Synthesis: Utilize LaTeX generation tools to compile a final PDF report including the generated graph, experimental methodology, and derived conclusions.